Two-sided assembly line balancing that considers uncertain task time attributes and incompatible task sets

Article *in* International Journal of Production Research · February 2020

DOI: 10.1080/00207543.2020.1724344

CITATIONS
15

READS
276

3 authors, including:

Yuchen Li
Beijing University of Technology
28 PUBLICATIONS **439** CITATIONS
SEE PROFILE

Ibrahim Kucukkoc
University of Exeter
82 PUBLICATIONS **2,229** CITATIONS
SEE PROFILE

International Journal of Production Research

ISSN: 0020-7543 (Print) 1366-588X (Online) Journal homepage: https://www.tandfonline.com/loi/tprs20

Two-sided assembly line balancing that considers uncertain task time attributes and incompatible task sets

Yuchen Li, Ibrahim Kucukkoc & Xiaowen Tang

To cite this article: Yuchen Li, Ibrahim Kucukkoc & Xiaowen Tang (2020): Two-sided assembly line balancing that considers uncertain task time attributes and incompatible task sets, International Journal of Production Research, DOI: 10.1080/00207543.2020.1724344

To link to this article: https://doi.org/10.1080/00207543.2020.1724344

Published online: 20 Feb 2020.

International Journal of Production Research, 2020
https://doi.org/10.1080/00207543.2020.1724344

Two-sided assembly line balancing that considers uncertain task time attributes and incompatible task sets

Yuchen Li[a], Ibrahim Kucukkoc [b*] and Xiaowen Tang[a]

[a]*School of Economics and Management, Beijing University of Technology, Beijing, People's Republic of China;* [b]*Industrial Engineering Department, Balikesir University, Balikesir, Turkey*

(*Received 13 August 2019; accepted 24 January 2020*)

An assembly line is a serial production system that is meant to produce high-quality and usually complex products in mass quantities. Assembly lines play a crucial role in determining the profitability of a company, as they are utilised as the final stage of the system prior to shipping. In an assembly line balancing problem, the assembly tasks are allocated to workstations based on their processing times after considering the precedence relationships between them. There is a massive amount of research in the literature using deterministic task processing times, and many other works consider stochastic task times. This research utilises the uncertainty theory to model uncertain task times and considers incompatible task sets constraints. The problem is solved using a simulated annealing algorithm with problem-specific characteristics. Lower bounds are developed to accelerate the simulated annealing algorithm. A restart mechanism, which can escape the local optimum obtained by neighbourhood generation, is proposed. A repair mechanism is integrated to combine the workstations so as to further improve the quality of solutions. The numerical examples and experimental tests demonstrate the powerful solution-building capacity of the proposed simulated annealing algorithm over teaching–learning-based and genetic algorithms. The methodology proposed in this research is applicable to any industry (including the automotive industry) when the historical data on task processing times is very limited.

Keywords: metaheuristics; two-sided assembly line balancing; uncertainty theory; incompatible task sets; simulated annealing

1. Introduction

An assembly line is an important manufacturing process that uses machines to move material or parts from one place to another. Assembly lines are widely employed to produce various types of products, including automobiles, electronic products, and jewelry. The main components of a standard assembly line are a conveyor belt, operators, workstations, interchangeable parts and tasks (Li, Kucukkoc, and Tang 2019). Assembly lines can be divided into one-sided (OAL) and two-sided lines (TAL) according to the task partitions, see Battaïa and Dolgui (2013) and Cerqueus and Delorme (2019). The assembly line balancing problem (ALBP) is a classical combinatorial optimisation problem that falls into the NP-hard category. In this paper, we address the two-sided assembly line balancing problem (TALBP).

The layout for a two-sided assembly line is shown in Figure 1. The workstations are located on both sides of the line. A pair of opposite stations, such as station 1 and station 2, is called a mated station, and one station calls the other one a companion. Operators perform the tasks for one part of the assembly process at the same mated station but different workstations simultaneously. The tasks can be classified into three groups: left (L), right (R), and either (E). Whereas tasks in groups L and R can only be assigned to the left and right workstations, respectively, tasks in group E can be assigned to either side of the assembly line. Compared with one-sided lines, two-sided lines have the following advantages: (1) the line length can be shortened so that fewer workers are required; (2) the throughput time can be reduced; (3) the cost of production is lowered because the tools and fixtures are shared by both sides of the mated-stations; and (4) the material handling, worker movements and set-up time are reduced.

Bartholdi (1993) was the first to explore TALBP, and he proposed some theoretical properties and developed a first-fit algorithm for task assignment. Kim, Kim, and Kim (2000) developed a genetic algorithm to minimise the number of stations for TALBP. Hu et al. (2010) proposed a branch-and-bound algorithm to minimise the number of mated-stations for TALBP,

*Corresponding author. Email: ikucukkoc@balikesir.edu.tr

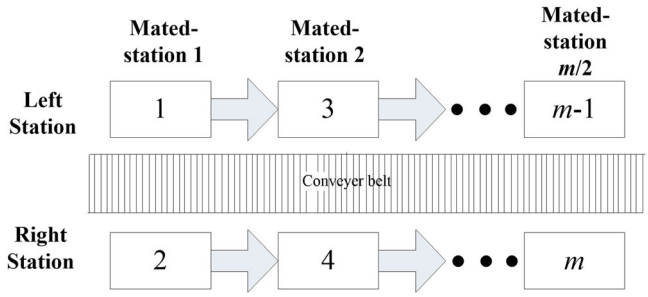

Figure 1. The basic structure of a two-sided assembly line.

and in this work, some efficient lower bounds were constructed. Agpak, Fatih Yegül, and Gökçen (2012) developed a 0-1 integer programming model to solve the two-sided U-type ALBP. More recently, Li et al. (2016a) proposed a teaching–learning-based algorithm to solve TALBP while considering multiple constraints. Li et al. (2016b) solved the mixed-model TALBP by a hybrid imperialist competitive algorithm. Li, Tang, and Zhang (2016) investigated the energy consumption objective in TALBP and proposed a restarted simulated annealing to solve it. Lei and Guo (2016) proposed a variable neighbourhood search method to minimise the cycle time for TALBP. Kucukkoc and Zhang (2016) solved the mixed-model parallel TALBP by an ant colony algorithm. Tang, Li, and Zhang (2016) presented a bee colony algorithm to minimise the cycle time for TALBP. Methods based on the particle swarm algorithm have been proposed for U-shaped TALBP (Delice et al. 2017b) and mixed-model TALBP (Delice et al. 2017a). Li, Tang, and Zhang (2017) developed an iterated greedy search algorithm for TALBP and demonstrated that the proposed algorithm outperformed other fourteen algorithms. A new workstation concept, called underground workstation, has been introduced by Kucukkoc et al. (2018) and the mixed-model TALBP was modelled mathematically considering underground workstations. An ant colony algorithm was also proposed by Kucukkoc et al. (2018) to solve the large-scale problems efficiently. Yang and Cheng (2019) proposed an effective variable neighbourhood search algorithm to solve TALBP with forward and backward set-up times. Li, Kucukkoc, and Zhang (2020) developed a branch, bound and remember algorithm which produces state-of-the-art results for two-sided assembly line balancing problems. There also are other examples of heuristics proposed for solving various types of line balancing problems. For example, Saif et al. (2017) proposed a hybrid pareto artificial bee colony algorithm to solve the multi-objective ALBP with task time variations. Zhang, Hu, and Wu (2018) addressed the two-sided assembly line re-balancing problem of a shovel loader and developed a modified multi-objective genetic algorithm. Zhang, Hu, and Wu (2019) utilised an improved imperial competition algorithm to solve the multi-objective TALBP with space and resource restrictions. Saif et al. (2019) proposed an artificial bee colony algorithm for order-oriented simultaneous sequencing and balancing of multi-mixed model assembly lines. For a comprehensive review of the objectives and algorithms in TALBP, readers can refer to Li, Kucukkoc, and Nilakantan (2017).

1.1. Uncertain task times and uncertainty theory

The majority of the research in ALBP is based on the presumption of deterministic task times. However, the task times are sometimes uncertain in real-world applications. Task time uncertainty may result from instability of the operator's work rates, the varied skills and motivations of workers, and the failure sensitivity of complex processes' uncertainty (Becker and Scholl 2006). The fundamental optimisation methods to handle uncertainties for ALBP are stochastic programming, stability analysis, and robust optimisation (see for example, Sotskov, Dolgui, and Portmann 2006; Sotskov et al. 2015; Lai et al. 2016; Lai, Sotskov, and Dolgui 2019.) They are applicable in different assumptions and serve different purposes. The stochastic programming can be utilised when the historical data for task times are adequate. The probability distributions can be estimated from the data. When the task times are known only within certain bound, robust optimisation can be used to estimate the worst-performance of the assembly line (Hazır and Dolgui 2012). When no historical information is involved, stability analysis can be employed to probe into the impact of uncertain task times (Gurevsky, Battaïa, and Dolgui 2012). However, the stability analysis is not very mathematical tractable. When there is a lack of data, how to handle uncertainties in an analytical tractable sense is the main scope of our paper.

While a copious amount of research has addressed uncertain task times for a one-sided line (see Sivasankaran and Shahabudeen 2014), very few scholars have addressed a two-sided line while considering the task time uncertainty. Özcan (2010) was the first to investigate TALBP given stochastic task time distributions and to develop a simulated annealing algorithm to solve the problem. Later, Chiang, Urban, and Luo (2015) utilised joint probabilities to measure the task time uncertainty and constructed a particle swarm algorithm. Delice, Aydogan, and Özcan (2016) employed a genetic algorithm

to solve the U-shaped TALBP with stochastic task times. Tang et al. (2017) balanced the TALBP with stochastic task times using a teaching–learning-based algorithm. In the literature, the task time uncertainty has always been modelled by probability theory (stochastic programming), and the task times are considered as random variables. In this paper, we address TALBP with uncertain task times by a new theory, i.e. uncertainty theory.

Uncertainty theory was proposed by Liu (2007) to model the belief degrees of experts. It has become a new branch in mathematics for gauging the indeterminate phenomena. Later, Liu (2009b) developed an uncertain programming model pertaining to the uncertain variables. Since then, uncertainty theory has been widely employed to model the uncertain inputs of various optimisation problems, such as facility location allocation (Wen, Qin, and Kang 2014), project scheduling (Ke, Liu, and Tian 2015), bicriteria solid transportation (Lin, Jin, and Bo 2017), optimal control (Li and Zhu 2016), vehicle routing (Ning and Su 2017), machine scheduling (Li and Liu 2017), and minimum spanning trees (Gao and Jia 2016). To be specific, in our research, we use uncertainty theory to model the belief degrees about the uncertain tasks times, which is similar to what was done in the work previously discussed, e.g. in facility location allocation problem, the belief degrees about the uncertain demand were modelled by uncertainty theory. The fundamentals of uncertainty theory and uncertain programming are described in the appendix.

Uncertainty theory and probability theory can both be utilised to model indeterminate event and perform mathematical tractable analysis. The probability theory is quite useful when the distribution of past data or information is very close to the objective *frequency*. For example, we flip a coin 100 times, the percentages that each number will occur is approximately 50%. In this case, we can estimate the likelihood of the next number by probability theory as the coin is flipped again. Therefore, one can fit a theoretical distribution with adequate historical data. However, sufficient data and information may be unavailable under low-volume production for mass-customised items and one-of-a-kind production for large-scale projects (Boysen, Fliedner, and Scholl 2008). For instance, one may not have any prior data before the production process when manufacturing a novel aircraft for a special purpose (as there is no time to run the pilot production and the cost to do that would be extremely high). Therefore, we cannot obtain the frequencies of the task times by analysing the past samples. Instead, we can only predict it from our subjective mind (*belief*). For each possible operation time, there is a belief degree in our mind which describes the likelihood the specific time will be the real one. The belief degree depends heavily on the personal knowledge concerning the event. Therefore, different people have different belief degrees for the same event. In this paper, we adopt the uncertainty theory to model belief degrees of the task times.

As discussed previously, uncertainty distributions are required to be known prior to the mathematical modelling. How to process the experts' beliefs into uncertainty distributions is out of the scope of the paper. In a nutshell, there are two approaches to collect the experts' belief data. The first approach is to conduct a questionnaire survey about the task times (Liu 2010). The second approach is to employ the so-called 'uncertain simulation' (Zhu 2012). The first step is to generate the data by some known probability distributions. The next step is to derive the uncertainty distribution from these data by various methods, such as principle least-square, delphi method.

1.2. Discrepancies and contributions

Our work can be distinguished from the related works in the literature due to the following features. First, we handle the uncertain task time attribute from the new perspective of uncertainty theory, which is applicable when the historical data for the task times are scarce. Because the mathematical foundation of the uncertainty theory is different from other methods, our proposed model is novel compared with models in other studies. Second, we introduce the incompatible task constraint into the mathematical model. This task constraint was first applied to TALBP by Kucukkoc (2018). Incompatible tasks are those tasks that cannot be performed concurrently at the same mated station. Sometimes, modelling and solving TALBP with uncertain task times by probability theory is not desirable in real practice. From previous studies, we can find that there may not exist a feasible solution under a specific cycle time. These outcomes may cause difficulties for the management process of production scheduling since the planner is unable to find any feasible solution with the current input and requirement, which may delay the production. In the proposed model, the feasibility issue does not exist.

This paper is organised as follows: Section 2 presents the problem description and its mathematical formulation, and Section 3 provides a comprehensive explanation of the proposed simulated annealing method including some basic definitions of the task times and lower bounds. The results of the experimental tests are reported in Section 4 together with the parameter settings we used, and the paper is concluded in Section 5.

2. Problem formulation

In this section, we present the procedure of the proposed problem.

2.1. Problem set-up

We present the following necessary assumptions and conditions for the problem formulation.

- The travel times and set-up times are ignored.
- A task can be assigned to only one workstation.
- Tasks in the same mated station but in different workstations can be performed simultaneously.
- The task assignments must conform to the precedence relationships, which are known and deterministic.
- The positional constraint ensures that some tasks can only be assigned to the right or left workstations.
- Some task pairs that are assigned to the same mated station, but different workstations cannot be processed concurrently (the incompatible task constraint).
- The cycle time is provided, and the station's completion time cannot be greater than the cycle time.
- The task times are uncertain variables, and their uncertainty distributions are known and given.
- The goal is to minimise the number of mated stations as the primary objective and the number of workstations as the secondary objective.

The notations used throughout the paper are presented in Table 1.

2.2. Mathematical formulation

In this section, we delineate the mathematical model of the proposed problem. We put a '' above the task-time-related parameters, which are uncertain variables. The task-processing-time distributions, precedence relationships, incompatible task matrix, cycle time and direction requirements for the tasks are given as input into our model. We utilise the chance constraint to control the situation in which the finishing time of the last task performed at a workstation is not greater than the cycle time. As stated by Hu and Wu (2018), in two-sided assembly lines, the workload of a workstation is possibly lower than the finishing time of the last task performed in that workstation. That is due to the idle time windows usually caused by the precedence relationships between tasks performed on the opposite sides of the line (Hu and Wu 2018). Note that chance-constrained programming was invented by Charnes and Cooper (1961). This programming is used to achieve certain objectives with the given confidence level α.

Table 1. Notations.

Variable names	Descriptions
c	The cycle time of the line
k	$k = 1$ indicates the left (side) workstation; $k = 2$ indicates the right (side) workstation
n	The number of tasks
t_i	The task processing time for task i, $i = 1, 2, \ldots, n$
t_i^s	The starting time for task i, $i = 1, 2, \ldots, n$
w_1, w_2	The weights associated with the total number of mated stations and the total number of workstations, respectively
z_{ijk}	$z_{ijk} = 1$ if task i is assigned to mated station j and workstation k; otherwise, $z_{ijk} = 0$
α	The confidence level
A_j/B_j	1 if the left or right workstation at mated station j is open; 0 otherwise
$D(i)$	The direction for task i. $D(i) = 1$ indicates that i should be assigned to the left side of the mated station, $D(i) = 2$ indicates that i should be assigned to the right side of the mated station, and $D(i) = 0$ indicates that i can be assigned to either side of the mated station
NM	The total number of mated stations
WS	The total number of workstations
I	The incompatible task matrix, where $I(v, b) = 1$ if and only if task v and task b cannot be performed concurrently at the different workstations of the same mated station
P	The direct precedence relationship matrix, where $P(v, b) = 1$ if and only if task v is a direct predecessor of task b
Q_{jk}	The set of tasks that is assigned to workstation k of mated station j
G	A large number
$\Phi_i(x)/\Phi_i^{-1}(\alpha)$	The uncertainty/inverse distribution for the processing time of task i
$\Psi_i(x)/\Psi_i^{-1}(\alpha)$	The uncertainty/inverse distribution for the starting time of task i
$\Upsilon_i(x)/\Upsilon_i^{-1}(\alpha)$	The uncertainty/inverse distribution for the finishing time of task i

2.2.1. Objective function and decision variables

In a two-sided assembly line, minimising the number of mated stations is more important than minimising the number of workstations. This is because the number of mated stations utilised on a line corresponds to the length of that line. From the practical view, it is favoured to have a shorter line instead of a long line on which only one workstation is utilised in each mated station. To maintain this aim, the objective function of our problem is a weighted sum of the number of mated stations and workstations:

$$\text{Obj} = w_1 \text{NM} + w_2 \text{WS},$$

where $w_1 \gg w_2$.

The decision variables are z_{ijk}, t_i^s, A_j, and B_j. z_{ijk} determines the task assignment at workstation k of mated station j, t_i^s is the starting time for task i, and A_j and B_j are indicator variables that show whether the left and right workstations at mated station j are open, respectively (if a station is 'open', it means tasks can be assigned to it).

2.2.2. Constraints

Here, we establish the constraints. The first constraint indicates the calculation of WS.

$$\text{WS} = \sum_{j=1}^{\text{NM}} (A_j + B_j). \tag{1}$$

The second constraint is the task occurrence constraint, as a task can only be assigned to one workstation and one mated station.

$$\sum_{k=1}^{2} \sum_{j=1}^{\text{NM}} z_{ijk} = 1, \quad \forall\, i = 1, 2, \ldots, n. \tag{2}$$

The third constraint is the chance constraint that guarantees that the product uncertain measure (see Definition 1 in the appendix) for *all* of the stations' completion times is less than or equal to the cycle time, which must itself be greater than or equal to α. Assume task q is the last task at station k of mated station j. Then, we let Λ_{jk} represent the event in which the completion time of workstation k of mated station j is less than or equal to the cycle time, i.e. $\Lambda_{jk} = \{\tilde{t_q^s} + \tilde{t_q} \leq c,\ q \in Q_{jk}\}$. Therefore, the third constraint is

$$\mathcal{M}\left\{\prod_{j=1}^{\text{NM}} \prod_{k=1}^{2} \Lambda_{jk}\right\} \geq \alpha.$$

By Axiom 4 of uncertainty theory, the product uncertain measure is equal to the minimum of the individuals' uncertain measures. That is,

$$\mathcal{M}\left\{\prod_{j=1}^{\text{NM}} \prod_{k=1}^{2} \Lambda_{jk}\right\} = \bigwedge_{j=1}^{\text{NM}} \bigwedge_{k=1}^{2} \mathcal{M}_{jk}\{\Lambda_{jk}\} \geq \alpha.$$

Hence, the above inequality can be transformed into

$$\begin{aligned} &\mathcal{M}_{jk}\{\Lambda_{jk}\} \geq \alpha, \quad \forall\, j = 1, 2, \ldots, \text{NM},\ \ k = 1, 2, \\ &\quad \Leftrightarrow \mathcal{M}\{\tilde{t_q^s} + \tilde{t_q} \leq c\} \geq \alpha, \quad q \in Q_{jk},\ \ \forall\, j = 1, 2, \ldots, \text{NM},\ \ k = 1, 2. \end{aligned}$$

Because the finishing time of the last task q increases with $\tilde{t_q^s}$ and $\tilde{t_q}$, we transform the chance constraint into the crisp constraint according to Theorem 3,

$$\Psi_q^{-1}(\alpha) + \Phi_q^{-1}(\alpha) - c \leq 0, \quad q \in Q_{jk},\ \ \forall\, j = 1, 2, \ldots, \text{NM},\ \ k = 1, 2.$$

According to Theorem 2, $\Upsilon_q^{-1}(\alpha) = \Psi_q^{-1}(\alpha) + \Phi_q^{-1}(\alpha)$. Therefore, the third constraint becomes

$$\Upsilon_q^{-1}(\alpha) - c \leq 0, \quad q \in Q_{jk},\ \ \forall\, j = 1, 2, \ldots, \text{NM},\ \ k = 1, 2. \tag{3}$$

The fourth constraint ensures the precedence relationship when the two tasks are assigned to different mated stations:

$$\sum_{j=1}^{\mathrm{NM}} j z_{vjk} \le \sum_{j=1}^{\mathrm{NM}} j z_{bjk}, \quad \text{if } P(v,b)=1, \ \forall\, v,b=1,2,\ldots,n, \ k=1,2. \tag{4}$$

The fifth constraint ensures the precedence relationship when the two tasks are assigned to the same mated station:

$$\tilde{t}_v^s + \tilde{t}_v \le \tilde{t}_b^s + G\left(1-\sum_{k=1}^{2} z_{vjk}\right) + G\left(1-\sum_{k=1}^{2} z_{bjk}\right)$$
$$\text{if } P(v,b)=1, \quad \forall\, v,b=1,2,\ldots,n, \ j=1,2,\ldots,\mathrm{NM}. \tag{5}$$

When tasks b and v are assigned to the same mated station, the fifth constraint is effective, and the above inequality will become

$$\tilde{t}_v^s + \tilde{t}_v \le \tilde{t}_b^s,$$

where the left-hand side is the finishing time of task v and the right-hand side is the starting time of task b.

Note that the fifth constraint guarantees that the finishing time of task v is less than or equal to the starting time of task b. The sixth and seventh constraints correspond to the incompatible task constraint; it should be noted that *only* one of them needs to be satisfied. Whereas the sixth constraint describes the situation in which the finishing time of task v is less than or equal to the starting time of task b, the seventh constraint describes the situation in which the finishing time of task b is less than or equal to the starting time of task v. The two constraints are as follows:

$$\tilde{t}_v^s + \tilde{t}_v \le \tilde{t}_b^s + G(1-z_{vjk}) + G(1-z_{bjk'})$$
$$\text{if } I(v,b)=1, \quad \forall\, v,b=1,2,\ldots,n, \ j=1,2,\ldots,\mathrm{NM}, \ k\neq k'. \tag{6}$$

$$\tilde{t}_b^s + \tilde{t}_b \le \tilde{t}_v^s + G(1-z_{vjk}) + G(1-z_{bjk'})$$
$$\text{if } I(v,b)=1, \quad \forall\, v,b=1,2,\ldots,n, \ j=1,2,\ldots,\mathrm{NM}, \ k\neq k'. \tag{7}$$

Now, the sixth/seventh constraint guarantees that the finishing time of task v/b is less than or equal to the starting time of task b/v. Finally, the eighth and ninth constraints are prescribed according to the positional requirements.

$$\sum_{j=1}^{\mathrm{NM}} z_{ij1} = 1, \quad \text{if } D(i)=1, \ \forall\, i=1,2,\ldots,n, \tag{8}$$

$$\sum_{j=1}^{\mathrm{NM}} z_{ij2} = 1, \quad \text{if } D(i)=2, \ \forall\, i=1,2,\ldots,n. \tag{9}$$

2.2.3. *Feasibility*

The existence of a feasible solution is very crucial for a mathematical model, especially when the model is applied in practice. For the models in Chiang, Urban, and Luo (2015) and Tang et al. (2017), there may not exist a feasible solution under some combinations of the cycle time and α (see Proposition 2 in Chiang, Urban, and Luo 2015). The reason is that their models require the joint probability of all stations' completion times to be greater than or equal to α. The joint probability is equal to the product of the marginal probabilities, which is the probability of one station's completion time being less than or equal to α. In some cases, especially in cases with large WS, this value may be very small, which would cause infeasibility. However, given the largest $\Phi_i^{-1}(\alpha)$ ($\forall\, i=1,2,\ldots,n$) is less than or equal to c, we can always find a feasible solution to the proposed model. To show that, we can simply assign exactly one task to one workstation, and the chance constraint becomes

$$\Phi_i^{-1}(\alpha) \le c, \quad \forall\, i=1,2,\ldots,n$$

Therefore, if the largest $\Phi_i^{-1}(\alpha),\ \forall\, i=1,2,\ldots,n$ is less than or equal to c, the chance constraint is satisfied. In the next section, we will derive inverse distributions for the task starting and finishing times, which are contingent upon the inverse distributions of the task processing time and the task sequence.

3. Solution procedure

3.1. The starting and finishing time of a task

Because the task processing time is an uncertain variable, the starting time of a task is an uncertain variable as well. Consider a typical TAL from Kim, Song, and Kim (2009) and the task assignment for the first mated-station in Figure 2. The information in parentheses above each task indicates the uncertain task time distribution ($\Phi(x)$) and the positional group (L, R, E). Tasks 1 and 3 are incompatible tasks, and they cannot be performed at the same mated station concurrently. We now derive the starting time of tasks for the first mated station. Tasks 1, 4 and 5 are assigned to the left station, and tasks 2, 3 and 7 are assigned to the right station. Inasmuch as tasks 1 and 2 are the first tasks performed at the left and right stations, respectively, their starting times are $t_1^s = 0$ and $t_2^s = 0$. Task 3 shall be performed after task 1 and 2 are finished, as task 1 is its incompatible task and task 2 is assigned prior to it at the same station. Thus, $t_3^s = \max(t_1^s + t_1, t_2^s + t_2)$. Task 4 can be performed immediately after task 1 is completed, as task 1 is task 4's predecessor. Thus, $t_4^s = t_1^s + t_1$. Task 5 can be performed immediately after both task 2 and task 4 are completed, so $t_5^s = \max(t_2^s + t_2, t_4^s + t_4)$. Task 7 can be performed when tasks 3 and 4 are finished, and we have $t_7^s = \max(t_3^s + t_3, t_4^s + t_4)$. Likewise, the starting time of tasks for the second mated station can be derived.

From the example, it can be deduced that a task's starting time depends on the *finishing times* of three types of tasks: (1) its prior task on the same side of the mated station; (2) its predecessors on the other side of the mated station; and (3) its incompatible tasks on the other side of the mated station. The three types of tasks can be grouped into a task set denoted as *Ts*. Therefore, the starting time of task i is

$$t_i^s = \max_{l \in Ts}(t_l^s + t_l)$$

Furthermore, the starting time of task i is an increasing function of the task's starting time t_l^s and processing time t_l. As a result, we can utilise Theorem 2 to derive the inverse uncertainty distribution for the starting time of task i.

$$\Psi_i^{-1}(\alpha) = \max_{l \in Ts}(\Psi_l^{-1}(\alpha) + \Phi_l^{-1}(\alpha)) \tag{10}$$

The finishing time for task i can also be derived as follows:

$$\Upsilon_i^{-1}(\alpha) = \max_{l \in Ts}(\Psi_l^{-1}(\alpha) + \Phi_l^{-1}(\alpha)) + \Phi_i^{-1}(\alpha) \tag{11}$$

It should be noted that Equation (11) can be used to calculate the station time when task i is the last task assigned to the station.

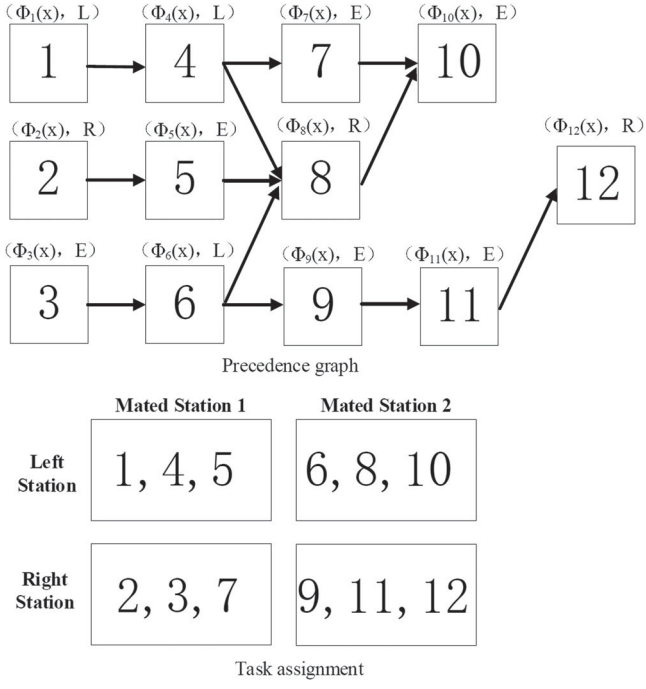

Figure 2. An illustrative example for the starting time of tasks.

The inverse distributions for the starting and finishing times of the tasks can be computed progressively as the tasks are assigned by the above formula. Compared with the probability distribution, which can only be approximated by the Clark approximation method in Chiang, Urban, and Luo (2015) and Tang et al. (2017), the proposed uncertainty distribution can provide a precise calculation for the completion time of a station.

The above formula facilitates the algorithm's construction because some constraints are satisfied automatically. Constraint (5) is satisfied because the formula considers the precedence relationships, which is the second type of task in *Ts*. Constraint (6)/(7) is satisfied because *Ts* contains the third type of task, which corresponds to the incompatible task relationships.

3.2. The lower bounds

In this section, we develop the lower bounds for NM and WS. Firstly, we consider the lower bound for WS. One extreme scenario is that no idle time is incurred at any workstation and the '=' sign in constraint (3) is effective for all of the stations, i.e. $\Upsilon_q^{-1}(\alpha) = \Psi_q^{-1}(\alpha) + \Phi_q^{-1}(\alpha) = c$. Because there is no idle time in the middle of a station, the starting time for a task is equal to its prior task's finishing time at the same station. Similar to the lower bound development with a deterministic task time in Hu, Wu, and Jin (2008), we combine the stations into one station to compute the lower bound and apply the above equation:

$$\sum_{i=1}^{n} \Phi_i^{-1}(\alpha) = \mathrm{WS} \cdot c, \quad \mathrm{LB}_1^{\mathrm{WS}} = \left\lceil \sum_{i=1}^{n} \Phi_i^{-1}(\alpha)/c \right\rceil.$$

In some cases, it is likely that the ceilings of the values of the left and right workstations will exceed $\mathrm{LB}_1^{\mathrm{WS}}$, especially when most of the tasks can be assigned to only one side of the mated stations. Thus, we have the second lower bound

$$\mathrm{LB}_2^{\mathrm{WS}} = \left\lceil \sum_{i \in S_L} \Phi_i^{-1}(\alpha)/c \right\rceil + \left\lceil \sum_{b \in S_R} \Phi_b^{-1}(\alpha)/c \right\rceil,$$

where S_L and S_R are the task sets containing the tasks that can only be assigned to the left or right side, respectively. Furthermore, the lower bound for WS can also be related to the longest path through the precedence graph. We can assign tasks to stations until the value $\sum_{i \in Ph^l} \Phi_i^{-1}(\alpha)$ exceeds the cycle time and a new station is open, where Ph^l is a path in the precedence graph. Therefore, the third lower bound is the maximum number of stations required for all of the paths in the precedence graph:

$$\mathrm{LB}_3^{\mathrm{WS}} = \max_l \{Z_l\},$$

where Z_l is the number of stations that can contain the tasks from path l. Hence, the lower bound for WS is the maximum value of the above three lower bounds:

$$\mathrm{LB}^{\mathrm{WS}} = \max\left\{\mathrm{LB}_1^{\mathrm{WS}}, \mathrm{LB}_2^{\mathrm{WS}}, \mathrm{LB}_3^{\mathrm{WS}}\right\}.$$

Now, we turn our attention to the lower bound of NM. Firstly, when the task times can be evenly split between the left and right workstations,

$$\mathrm{LB}_1^{\mathrm{NM}} = \left\lceil \mathrm{LB}_1^{\mathrm{WS}}/2 \right\rceil.$$

Secondly, we can consider two sides individually as follows:

$$\mathrm{LB}_2^{\mathrm{NM}} = \max\left\{ \left\lceil \sum_{i \in S_L} \Phi_i^{-1}(\alpha)/c \right\rceil, \left\lceil \sum_{j \in S_R} \Phi_j^{-1}(\alpha)/c \right\rceil \right\}.$$

Finally, the longest path that goes through all of the mated stations can also imply the lower bound for NM ($\mathrm{LB}_3^{\mathrm{WS}}$). Hence, the lower bound for NM is

$$\mathrm{LB}^{\mathrm{NM}} = \max\left\{\mathrm{LB}_1^{\mathrm{NM}}, \mathrm{LB}_2^{\mathrm{NM}}, \mathrm{LB}_3^{\mathrm{WS}}\right\}.$$

Considering our objective function, the lower bound for our objective value is

$$\mathrm{LB}^{\mathrm{Obj}} = w_1 \mathrm{LB}^{\mathrm{NM}} + w_2 \mathrm{LB}^{\mathrm{WS}}. \tag{12}$$

3.3. Task dominance

In this section, we extend the task dominance relationship for one-sided ALBP to a sequence-dependent task dominance relationship specific to our problem. Task i dominates task j if the following criteria are met: (1) two tasks are not related regarding precedence; (2) two tasks can be assigned to the same side of a station; (3) the set of successors of task j is a subset of the set of successors of task i, i.e. $\text{Suc}_j \subseteq \text{Suc}_i$; (4) the inverse uncertainty distribution of the finishing time of task i is not less than that of task j, i.e. $\Upsilon_i^{-1}(\alpha) \geq \Upsilon_j^{-1}(\alpha)$. If $\Upsilon_i^{-1}(\alpha) = \Upsilon_j^{-1}(\alpha)$ and $\text{Suc}_i = \text{Suc}_j$, then i dominates j for $i < j$. The task dominance rule is sequence-dependent since the finishing time of a task is influenced by the starting time of previous assigned tasks (c.f. Section 3.1). The dominance relation can improve the performance of SA. It exploits more remaining capacity of a station which, in return, reduces the remaining unassigned loads for the subsequent stations (see Section 3.8).

3.4. The overview of SA

SA is an optimisation method which mimics the slow cooling process of metals. It has become a popular local search meta-heuristic to solve both discrete and continuous optimisation problems since it was introduced by Kirkpatrick, Gelatt, and Vecchi (1983). Khorasanian, Hejazi, and Moslehi (2013), Dong et al. (2014) and Roshani and Giglio (2016) developed SA approaches to solve ALBPs successfully. The success of SA mainly arises from its stochastic search mechanism to escape local optima by allowing hill-climbing moves. SA begins with an initial solution S, and a new solution S' is generated by the neighbourhood generation method. S and S' correspond to different fitness values, i.e. Ft and Ft', respectively. For a minimisation problem, if the amount of change in the fitness value (i.e. $\Delta = Ft' - Ft$) is lower than 0, then the new solution S' is accepted. If not, S' is accepted with a specified probability $exp^{-\Delta/T}$, where T is a controlled parameter that refers to the physical temperature. T is reduced by a cooling schedule in each iteration and this cycle is repeated until a predefined stopping criterion is met.

3.5. The general framework of SA

In SA, T_0, cr, $\text{iter}_{\max}$, $d_{\max}$, LB^{Obj}, c, α, and $\Phi_i^{-1}(\alpha)$ represent the initial temperature, cooling rate, maximum iteration, restart limit, lower bound for the objective values, cycle time, confidence level and inverse distribution for the task times, respectively. They are the input for the algorithm. The linear cooling schedule is chosen to reduce the temperature after each iteration. The structure of SA is demonstrated in Figure 3. The steps of SA are given iteratively below.

Step 1: Set the parameters T_0, cr, $\text{iter}_{\max}$, $d_{\max}$, c, α, $\text{iter} = 1$, $\Phi_i^{-1}(\alpha)$ and LB^{Obj}.

Step 2: Set $d = 1$, launch the initial solution generation process to obtain the initial solution S_0.

Step 3: Generate the initial objective value Obj_0 and fitness value Ft_0 from S_0 by a feasible task assignment.

Step 4: Set the temperature $T = T_0$; set the current solution S_C and the best solution S_{best} equal to S_0; set the current fitness value Ft_C to Ft_0; and set the best objective value Obj_{best} equal to Obj_0.

Step 5: Launch the neighbourhood generation process to obtain a new solution S_{iter}, its corresponding fitness value Ft_{iter} and its objective value Obj_{iter}, $\text{iter} = \text{iter} + 1$.

Step 6: Launch the repair algorithm to update S_{iter} and Ft_{iter}.

Step 7: Calculate $\Delta = Ft_{\text{iter}} - Ft_C$.

Step 8: If $\Delta \leq 0$, then accept the solution as the new current solution $S_C = S_{\text{iter}}$, $Ft_C = Ft_{\text{iter}}$, and go to step 9; otherwise, go to step 10.

Step 9: Check whether the objective value Obj_{iter} is equal to the lower bound LB^{Obj}. If equality is obtained, stop; otherwise, go to step 11.

Step 10: Generate a random number rn from an uniform distribution (0,1). If $rn \leq \exp^{-\Delta/T}$, accept the solution as the new current solution, set $S_C = S_{\text{iter}}$ and $Ft_C = Ft_{\text{iter}}$, and go to step 9; otherwise, S_C and Ft_C remain unchanged, and go to step 12.

Step 11: If $\text{Obj}_{\text{iter}} < \text{Obj}_{\text{best}}$, $S_{\text{best}} = S_{\text{iter}}$ and $\text{Obj}_{\text{best}} = \text{Obj}_{\text{iter}}$, go to step 12; otherwise, go to step 12.

Step 12: If $\text{iter} = \text{iter}_{\max}$, stop; otherwise, update $T = T * cr$, $\text{iter} = \text{iter} + 1$, $d = d + 1$, and go to step 13.

Step 13: If $d = d_{\max}$, go to step 2; otherwise, go to step 5.

3.6. The solution encoding and initial solution generation

In SA, we represent a solution with a sequence of priority score values. Tasks with higher scores are selected for assignment earlier than tasks with lower scores. We adopt the methods in Tuncel and Aydin (2014) to generate the initial priority scores

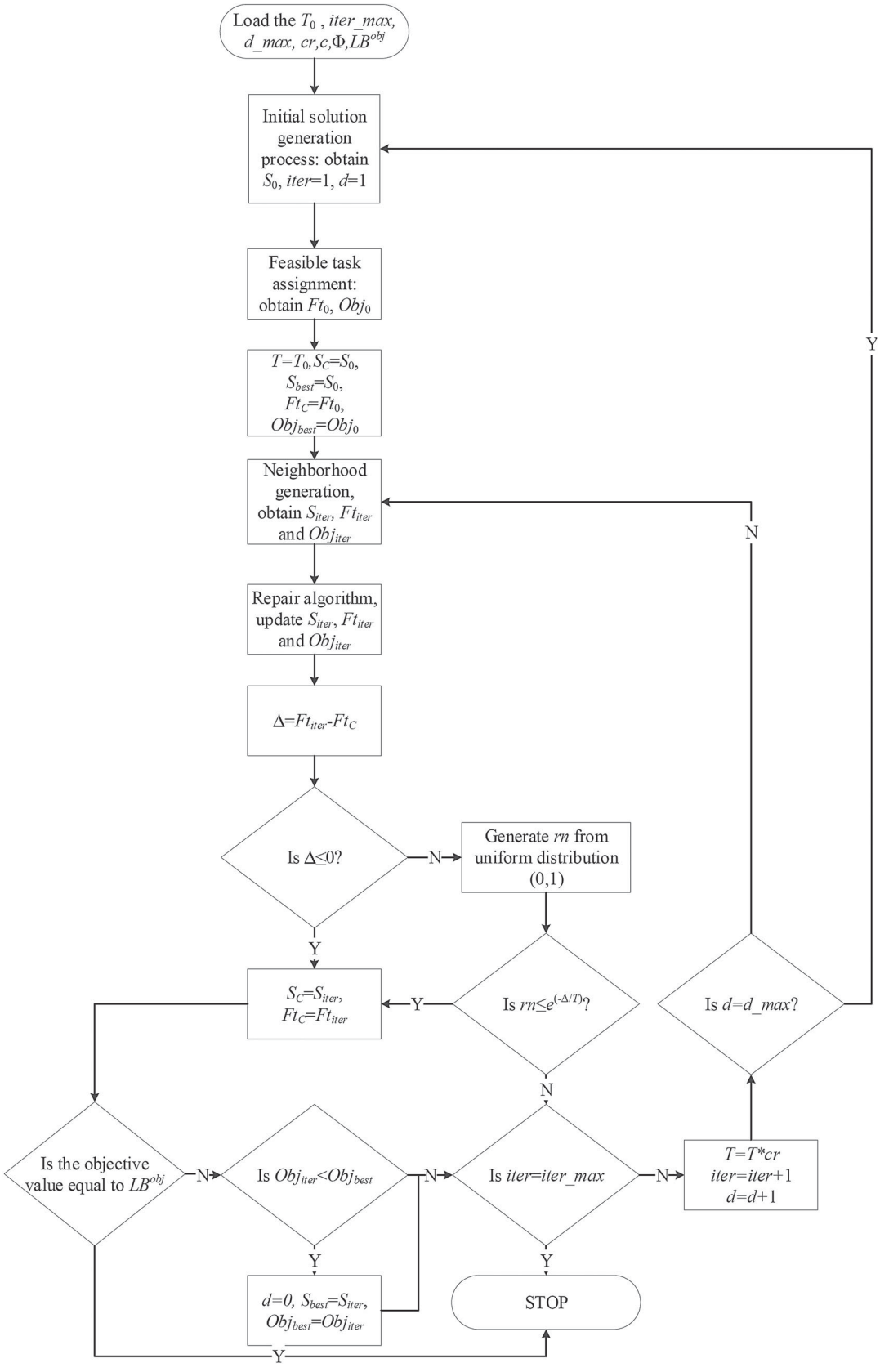

Figure 3. The general framework of SA.

as follows:

$$Pl_i = 1 + \text{random}[0,1] \times n,$$

where Pl_i is the priority score for task i and random[0,1] is a uniformly generated number that ranges from 0 to 1. By combining Pl with the task precedence relationship, the task assignment order is determined, and so are the starting and finishing times. In other words, we do not need to examine the precedence constraint in the algorithm. Next, we introduce the feasible task assignment process to convert the solution to the fitness value and objective value.

3.7. Fitness function

The fitness function determines whether a new solution is accepted or not. We modified the fitness function introduced in Li, Tang, and Zhang (2017) by replacing the deterministic station time by the inverse uncertainty distribution

$$Ft = \sum_{j=1}^{\text{NM}} \sum_{k=1}^{2} (\text{NM} + 1 - j) \times (c - \Upsilon_q^{-1}(\alpha)), \tag{13}$$

where q is the last task finished at workstation k of mated station j, and $\Upsilon_q^{-1}(\alpha)$ is the station time (j,k) in an uncertain environment given confidence level α. The motivation for choosing the fitness function is to encourage the former mated stations to undertake a greater workload. As such, it is highly probable that the workstations in the last mated station can be combined, which would reduce WS. The method to reduce the workstation number will be discussed in Section 3.9 (the repair algorithm).

3.8. Generation of a feasible task assignment

The following notation is used to describe the procedure for generating a feasible task assignment from a solution.

U	Unassigned task set
$\|U\|$	The number of unassigned tasks
CS	Current mated station
$Q_{\text{cs}}^L / Q_{\text{cs}}^R$	task assignment for left/right station of station CS.
LST/RST	Left/Right station time, which is equal to the finishing time of the last task assigned to the left/right workstation

The structure of the task assignment procedure is demonstrated in Figure 4.

The steps of the feasible task assignment are given iteratively below.

Step 1: Input S, and set CS $= 1$

Step 2: Set LST $=$ RST $= 0$, $U = S$, $Q_{\text{cs}}^L = Q_{\text{cs}}^R = \{\}$., and choose the first task in U to assign.

Step 3: If $D(i) = 1$, assign the task to the left workstation, update $Q_{\text{cs}}^L = \{Q_{\text{cs}}^L, i\}$ and go to step 8; otherwise, go to step 4.

Step 4: If $D(i) = 2$, assign the task to the right workstation, $Q_{\text{cs}}^R = \{Q_{\text{cs}}^R, i\}$ and go to step 8; otherwise, go to step 5.

Step 5: If RST $>$ LST, assign the task to the left workstation, $Q_{\text{cs}}^L = \{Q_{\text{cs}}^L, i\}$ and go to step 8; otherwise, go to step 6.

Step 6: If RST $<$ LST, assign the task to the right workstation, $Q_{\text{cs}}^R = \{Q_{\text{cs}}^R, i\}$ and go to step 8; otherwise, go to step 7.

Step 7: Assign the task to the left or right workstation with equal probability (by generating a random number), and go to step 8.

Step 8: Calculate RST or LST by Equation (11).

Step 9: If the station time is greater than c, undo the assignment of task i; otherwise, go to step 12.

Step 10: Substitute the tasks in Q_{cs}^L and Q_{cs}^R by their dominated tasks in U until no possible substitution exists to make the station time less than or equal to c, update U, Q_{cs}^L and Q_{cs}^R.

Step 11: Set CS $:=$ CS $+ 1$, and go back to step 2;

Step 12: Remove task i from U, and set $|U| := |U| - 1$.

Step 13: If $|U| = 0$, record Ft, and stop; otherwise, go to step 2.

By employing the above procedure, we can find the fitness value of a solution S. The objective value can also be obtained, i.e. NM $=$ CS, and WS is the number of stations that opened during the task assignment.

Example 1 Consider the precedence relationship graph given in Figure 5. We assume that the task time follows a normal uncertainty distribution (see Definition 2) as follows: $\mathcal{N}_1(1, 0.25)$, $\mathcal{N}_2(3, 0.75)$, $\mathcal{N}_3(1, 0.25)$, $\mathcal{N}_4(2, 0.5)$, $\mathcal{N}_5(3, 0.75)$, and

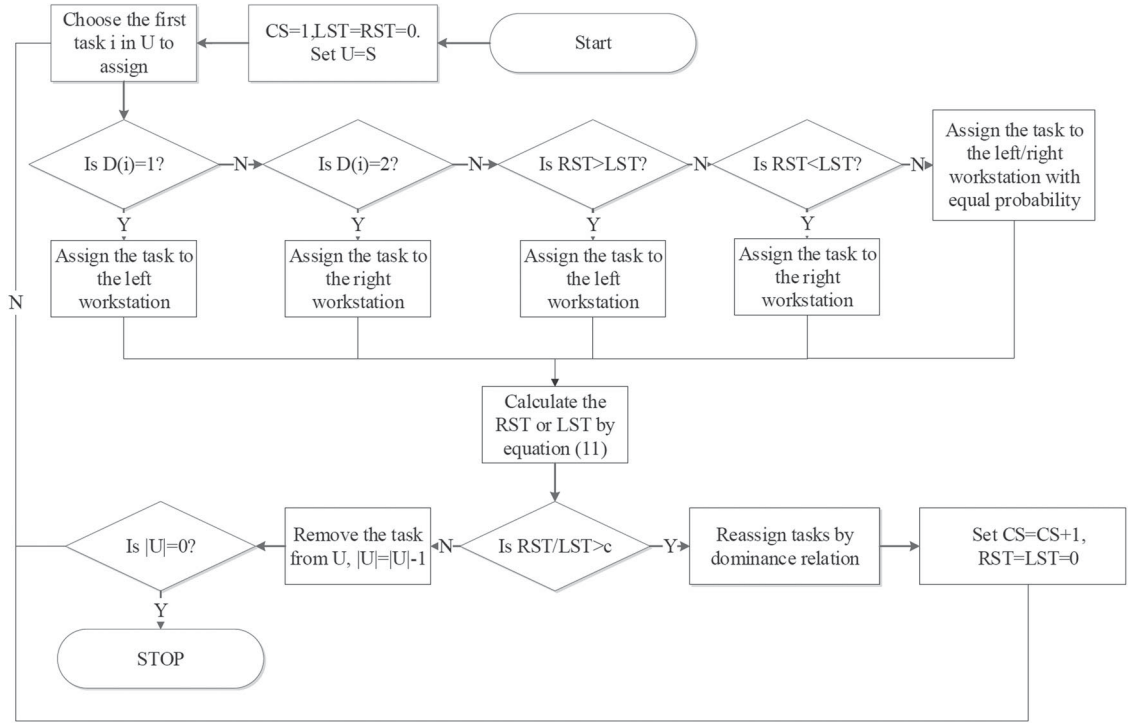

Figure 4. Task assignment procedure.

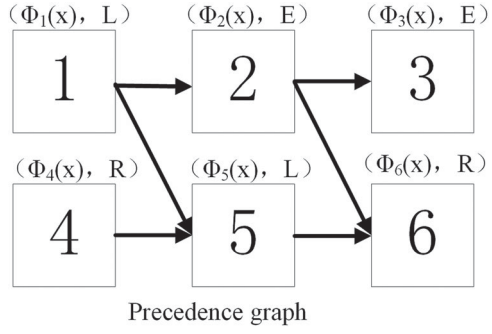

Figure 5. An illustrative example for the starting time of tasks.

$\mathcal{N}_6(2, 0.5)$. The inverse distribution under $\alpha = 0.9$ for the task time is $\Phi^{-1}(0.9) = \{1.3, 3.91, 1.3, 2.61, 3.91, 2.61\}$. Tasks 2 and 4 are incompatible tasks, and c is 5. The current solution is $S_C = \{4, 1, 2, 5, 3, 6\}$. We set CS = 1, $U = S_C$ and assign task 4 to the right station. According to Equation (11), RST = 2.61. Next, we update $U = \{1, 2, 5, 3, 6\}$ and assign task 1 to the left station LST = 1.3. Then, we update $U = \{2, 5, 3, 6\}$ and assign task 2 to the left station because LST < RST and RST = 6.52 (due to the incompatible task constraint, task 2 cannot be performed until task 4 is finished). Inasmuch as RST > c, we open a new position CS = 2, RST = LST = 0. We assign task 2 to the left station (assuming the left station is chosen in step 7), where LST = 3.91. Then, we update $U = \{5, 3, 6\}$ and assign task 5 to the left station, where LST = 7.82. Again, because LST > c, we open a new position CS = 3, RST = LST = 0. Task 5 is assigned to the left station LST = 3.91. We update $U = \{3, 6\}$ and assign task 3 to the right station RST = 1.3. We update $U = \{6\}$ and assign task 6 to the right station LST = 6.52. A new position is opened, CS = 4, RST = LST = 0. Finally, we assign task 6 to the right station RST = 2.61. The station configuration is illustrated in Figure 6. The shaded area shows the idle time for each station. NM = 4, WS = 6, and

$$Ft_C = 4(5 - 1.3) + 4(5 - 2.61) + 3(5 - 3.91) + 2(5 - 3.91) + 2(5 - 1.3) + (5 - 2.61) = 39.6.$$

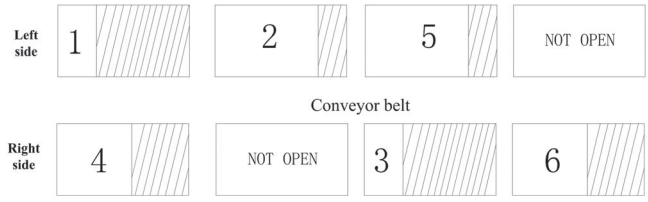

Figure 6. Station configuration under S_C.

3.9. Neighbourhood generation with restart mechanism

A new solution S_N can be generated from S_C by moving a task to another position. Firstly, a task i is randomly selected to be moved. Secondly, we find the positions of the last predecessor p_1 and first successor p_2 of task i in S_C. Then, the new position for task i must be selected between $p1$ and $p2$. The possible position is selected randomly. If there is no possible position, then another task is chosen, and the above procedure is repeated.

Example 2 Assume the setting is the same as in Example 1. We select task 5 to move. The last predecessor of task 5 is task 1, and the first successor of task 5 is task 6. Therefore, task 5 can be moved to the slot after task 1 and before task 6 without violating the precedence relationship constraint. We move task 5 before task 2 and set $S_N = \{4, 1, 5, 2, 3, 6\}$. Then, the new station configuration is illustrated in Figure 7. As a result, NM $= 3$, WS $= 6$, and $Ft_N = 28.72$. Compared with the current solution, the new solution improves NM by 1.

As is well known, neighbourhood search methods may lead to local optima. To address this issue, we can utilise the restart mechanism. Restart mechanism is a diversification strategy that can enlarge the global search space of SA. The mechanism is specified by a parameter $d_{\max}$. When the optimum is not improved in $d_{\max}$ consecutive iterations, the algorithm can go to the initial solution generation phase (c.f. Section 3.6).

3.10. Repair algorithm

The repair algorithm aims to combine the workstations at the last mated station so that WS can be reduced by one. The repair algorithm is invoked if the following criteria are met: (1) the two workstations at the last mated station are both open; (2) there is at least one station such that all of the tasks assigned to it are in group E; and (3) the sum of the inverse uncertainty distribution of the task times under a given α is not greater than c. If the above three criteria are met, we can reassign all of the tasks to one workstation by considering their precedence relationship.

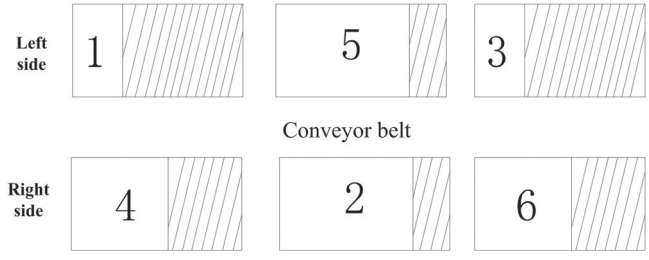

Figure 7. Station configuration under S_N.

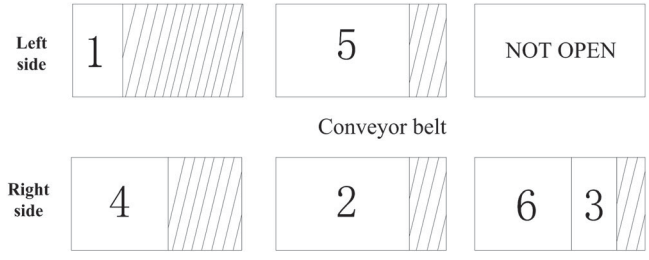

Figure 8. Station configuration by repair algorithm.

Example 3 Assume the setting is the same as in Example 2, where $S_N = \{4, 1, 5, 2, 3, 6\}$. Notice that task 3 belongs to group E and $\Phi_3^{-1}(0.9) + \Phi_6^{-1}(0.9) = 3.91 < 5 = c$. We can assign task 3 to the right workstation and close the left workstation, and the resulting new station configuration is illustrated in Figure 8. Therefore, NM = 3, WS = 5, and $Ft_N = 23.72$. Compared with the solution in Figure 7, the repair algorithm improves WS by 1.

4. Computational tests

4.1. Experimental settings

To test the performance of the proposed SA, three benchmark problems, P65, P148 and P205, are solved. We select these three datasets because medium-sized and large-sized problems are more convincing as benchmark problems to show an algorithm's efficiency. Whereas P65 and P205 were introduced by Lee, Kim, and Kim (2001), P148 is taken from Bartholdi (1993). Note that the processing times of task 79 and task 108 in P148 were changed to 111 and 43, respectively, by Kim, Kim, and Kim (2000). We further test the algorithms on more problems generated by SALBPgen (Otto, Otto, and Scholl 2013), which is a systematic data generator for ALBP. We report the results of these datasets in the appendix.

Two classical solution methods for solving the TALBP are selected for comparison: the genetic algorithm (GA) in Kim, Song, and Kim (2009) and the teaching–learning-based optimisation algorithm (TLBO) in Tuncel and Aydin (2014). We modify these two algorithms to solve the proposed problem by integrating the task time uncertainty and task incompatibility constraint. We implement these algorithms using Matlab (R2016a) and run it on an Intel Xeon E5606 2.13 Ghz CPU. The datasets and codes are provided in a supplementary file, and readers can reproduce the results by following the instructions.

From the preliminary experiments, we have observed that the optima are not sensitive to the parameter values. Therefore, we select the parameter values from the literature, as in Table 2. It should be noted that the 'crossover' action is performed in each iteration but the 'mutation' action is performed with some probabilities for GA. For the selection of the initial temperature in SA, we adopt the method in Vicente, Lanchares, and Hermida (2003). The initial temperature is chosen after considering the problem-specific characteristics. First, a few solutions are generated iteratively by the neighbourhood generation method. Then, the initial temperature is set to a value in such a way that most of the movements are accepted.

The experiments are conducted as follows. Firstly, we select the normal uncertainty distribution $\mathcal{N}(\mu, \sigma)$ for the task times. The original deterministic task time is set as the expected value of the task time (μ), and the variance of the task time σ is randomly generated between 0 and $(\mu/4)^2$ for a low-task variance and between 0 and $(\mu 2)^2$ for a high-task variance. There are two levels for the confidence interval α, $\{0.9, 0.975\}$. $w_1 = 10$ and $w_2 = 1$ indicate that NM is the primary objective to optimise. The algorithms are run 10 times in each case. The best objective values and the relative percentage deviations (RPD) are reported as $\text{RPD} = (\text{Obj} - \text{LB}^{(\text{Obj})})/\text{LB}^{(\text{Obj})} \times 100\%$. To compare the algorithms on an equal footing, we limit the algorithm runtime (rt) to $rt = n \times n$ milliseconds.

In Figure 9, we present a box plot to depict the performances of the three algorithms in terms of the RPD under different levels of α and variance. From the figure, we make two main observations: (1) SA outperforms the other two algorithms in terms of the median RPD, and (2) the output of SA exhibits lower variance than the outputs of the other two algorithms. Therefore, SA has a better distribution performance than the other two algorithms in solving the proposed problem. Furthermore, we select both ANOVA and the Friedman rank-based test to show the significance of the algorithms. The tests show that the p-values are close to zero in all cases, implying that there exists a significant difference in the average RPD and average rank values of the different methods.

Tables 3 and 4 summarise the algorithm comparisons on P65, P148 and P205 for the low-variance case and the high-variance case, respectively. We investigate 22 instances for each combination of α and the variance level. The unique best results are marked in bold. For the problems with low-task variance and $\alpha = 0.975$ (Table 3), the number of best solutions and unique best solutions generated by the SA are 21 and 13, respectively. The corresponding numbers for the GA and TLBO are (8,1) and (6,0), respectively. For the problems with low-task variance and $\alpha = 0.9$, the number of best solutions and unique best solutions generated by SA are 22 and 9, respectively. The respective corresponding numbers for GA and TLBO are (10,0) and (11,0). In this case, SA can find the best solution to every problem.

Table 2. Parameter values for each algorithm.

Parameters	GA	TLBO	SA
Population size	10	10	–
Mutation rate (solution)	0.2	–	–
Mutation rate (gene)	0.2	–	–
Cooling rate	–	–	0.99

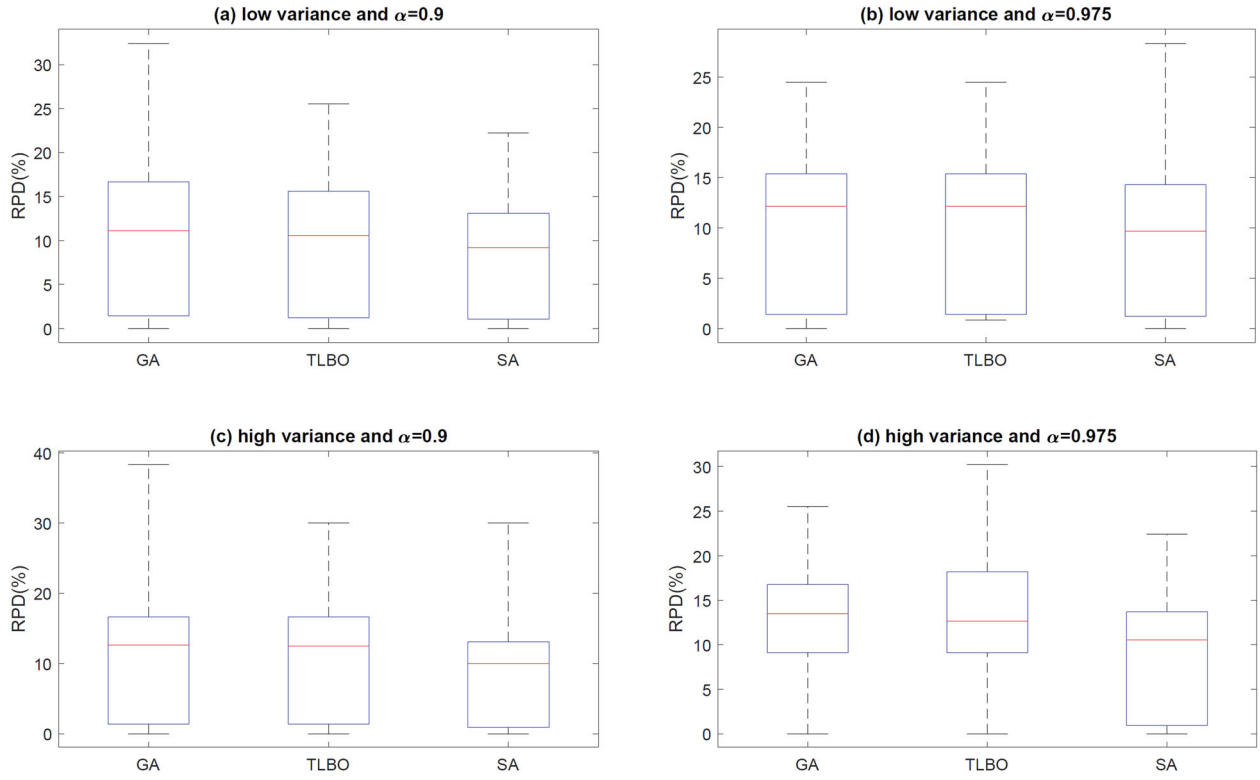

Figure 9. RPD under different levels of α and variance.

Table 3. Computational results on the test problems-low task variance.

			$\alpha = 0.975$				$\alpha = 0.9$		
Problem	c	LB	GA NM[WS]	TLBO NM[WS]	SA NM[WS]	LB	GA NM[WS]	TLBO NM[WS]	SA NM[WS]
P65	326	10[20]	**11[22]**	12[23]	12[23]	10[19]	11[21]	11[21]	11[**20**]
	381	9[17]	10[20]	10[19]	**9[18]**	8[16]	9[17]	9[18]	9[17]
	435	8[15]	8[16]	8[16]	8[16]	7[14]	8[15]	8[16]	8[15]
	490	7[14]	7[14]	7[14]	7[14]	6[12]	7[14]	7[13]	7[13]
	544	6[12]	7[14]	7[14]	**6[12]**	6[11]	6[12]	6[12]	6[12]
P148	204	16[32]	18[36]	18[36]	18[36]	15[30]	18[34]	17[34]	17[**32**]
	255	13[26]	15[29]	15[29]	**14[28]**	12[24]	13[26]	13[26]	13[**25**]
	306	11[22]	12[24]	12[24]	**11[22]**	10[20]	11[22]	11[21]	11[21]
	357	10[19]	10[20]	10[20]	10[**19**]	9[17]	9[18]	9[18]	9[18]
	408	8[16]	9[18]	9[18]	9[**17**]	8[15]	8[16]	8[16]	8[**15**]
	459	8[15]	8[16]	8[16]	8[**15**]	7[13]	7[14]	7[14]	7[14]
	510	7[13]	7[14]	7[14]	7[**13**]	6[12]	7[13]	7[13]	**6[12]**
P205	1133	13[26]	15[29]	15[29]	**14[28]**	12[24]	14[28]	14[27]	**13[26]**
	1322	11[22]	12[24]	13[25]	12[24]	11[21]	12[23]	11[22]	11[22]
	1510	10[20]	11[22]	11[22]	11[**21**]	9[18]	10[20]	10[20]	10[**19**]
	1699	9[18]	10[20]	9[18]	9[18]	8[16]	9[17]	9[17]	9[17]
	1888	8[16]	9[17]	9[17]	**8[16]**	8[15]	8[16]	8[16]	8[**15**]
	2077	7[14]	8[15]	8[16]	8[15]	7[13]	7[14]	7[14]	7[14]
	2266	7[13]	7[14]	7[14]	7[14]	6[12]	7[13]	7[13]	7[13]
	2454	6[12]	7[13]	7[13]	**6[12]**	6[11]	6[12]	6[12]	6[12]
	2643	6[11]	6[12]	6[12]	6[12]	6[11]	6[12]	6[12]	6[**11**]
	2832	6[11]	6[12]	6[12]	6[**11**]	5[10]	5[10]	5[10]	5[10]

Table 4. Computational results on the test problems-high-task variance.

			$\alpha = 0.975$				$\alpha = 0.9$		
Problem	c	LB	GA NM[WS]	TLBO NM[WS]	SA NM[WS]	LB	GA NM[WS]	TLBO NM[WS]	SA NM[WS]
P65	326	12[23]	14[27]	13[26]	13[26]	10[20]	12[23]	12[23]	12[24]
	381	10[19]	**11[22]**	12[23]	12[23]	9[17]	10[20]	10[20]	10[20]
	435	9[17]	10[19]	10[20]	10[19]	8[15]	9[17]	9[17]	9[17]
	490	8[15]	9[17]	9[17]	**8[16]**	7[14]	8[15]	8[16]	8[15]
	544	7[14]	8[16]	8[15]	8[15]	6[12]	7[14]	7[14]	7[**13**]
P148	204	19[37]	22[41]	22[41]	**21**[42]	17[33]	20[37]	20[38]	**18[36]**
	255	15[30]	**16**[32]	17[34]	17[32]	14[27]	15[30]	16[30]	15[**29**]
	306	13[25]	14[28]	14[28]	14[**27**]	11[22]	13[25]	13[25]	**12[24]**
	357	11[21]	12[24]	12[23]	12[23]	10[19]	11[21]	11[21]	**10[20]**
	408	10[19]	10[20]	10[20]	10[20]	9[17]	9[18]	9[18]	9[18]
	459	9[17]	9[18]	9[18]	9[18]	8[15]	8[16]	8[16]	8[**15**]
	510	8[15]	8[16]	8[16]	8[16]	7[14]	7[14]	7[14]	7[14]
P205	1133	16[31]	18[35]	18[35]	18[**34**]	14[27]	15[30]	16[31]	15[**29**]
	1322	13[26]	15[30]	15[30]	15[**29**]	12[23]	13[26]	13[26]	13[**25**]
	1510	12[23]	13[26]	13[26]	13[**25**]	10[20]	11[22]	11[22]	11[22]
	1699	11[21]	11[22]	11[22]	11[22]	9[18]	10[20]	10[20]	10[**19**]
	1888	10[19]	10[20]	10[20]	10[20]	8[16]	9[17]	9[18]	9[17]
	2077	9[17]	9[18]	9[18]	9[18]	8[15]	8[16]	8[16]	8[**15**]
	2266	8[16]	8[16]	8[16]	8[16]	7[14]	7[14]	7[14]	7[14]
	2454	7[14]	8[16]	8[16]	8[**15**]	7[13]	7[14]	7[14]	7[**13**]
	2643	7[13]	7[14]	7[14]	7[14]	6[12]	6[12]	6[12]	6[12]
	2832	7[13]	7[13]	7[13]	7[13]	6[11]	6[12]	6[12]	6[**11**]

For the problems with high-task variance and $\alpha = 0.975$ (Table 4), the numbers of best solutions and unique best solutions generated by SA are 20 and 7, respectively. The corresponding respective numbers for GA and TLBO are (12,2) and (12,0). For the problems with high-task variance and $\alpha = 0.9$, SA generates 21 best solutions and 12 unique best solutions, and the corresponding respective numbers for GA and TLBO are (10,0) and (8,0). Thus, SA outperforms the other two algorithms for both α values. Furthermore, the objective values are non-decreasing in terms of α and the variance. The variance reflects the level of uncertainty in the sense that the more uncertain the task time is, the more stations will be needed to satisfy the constraints. In addition, α can be interpreted as the stability of the assembly line system because it implies the likelihood that the station load is not greater than the cycle time. Hence, the larger α is, the more stable the assembly line will be, and the more stations will be required.

5. Conclusions

Uncertainty in the task time is very common in assembly line production. Almost all of the existing research focuses on modelling the task time uncertainty by probability theory, which requires data to obtain the task time distribution. However, there is a lack of data in some practical situations, especially when mass-producing customised items. The main objective of our paper is to characterise the two-sided assembly line balancing problem with the task time uncertainty using uncertainty theory, which is based on the experts' information. A chance-constrained mathematical formulation is developed that considers the incompatible task constraint, which was introduced in a recent study. In contrast to the stochastic optimisation model, where a feasible solution sometimes cannot be found, there is always a feasible solution to our model. Furthermore, a simulated annealing algorithm is proposed to solve the stated problem. Computational studies show that SA outperforms GA and TLBO in solving large-sized problems. Our research is the first attempt to model and solve the two-sided assembly line balancing problem with uncertainty theory. The methodology proposed in this research can be applied to major industries especially those incorporating the production of large vehicles and house appliances. Different from the probability theory, the application of the uncertainty theory is possible even when the historical data on the processing times of tasks are limited. In such situations, expert beliefs are put into practice to derive processing times of tasks. Moreover, the consideration of incompatible task sets makes it possible to reflect the real-world conditions in a two-sided assembly line. Different from the commonly applied negative zoning constraints (which restrict the assignment of conflicting tasks in the same workstation), incompatible task constraints prevent the concurrently processing of two tasks at the same mated-station.

That is more suitable to the production of large-sized products (such as trucks or vehicle cabins) which may allow only one operator to get on the product at the same time due to space limit.

There still are some limitations to this study. Although it is very promising, it is likely that the proposed algorithm is not the most efficient one for solving the problem. More comprehensive algorithm comparisons can be performed to further confirm SA's superiority. Moreover, this work does not compare the uncertain model and stochastic model in terms of the computational results. The other studies that apply uncertainty theory to other optimisation problems (see the literature review) cannot be compared because the assumptions for the problems are different. Even if we could apply the two theories to solving the same problem, we would still need to equalise the input distributions (the probability distribution and uncertainty distribution) of the models.

This paper sheds light on applying uncertainty theory to TALBP in future studies. Firstly, the type-II TALBP is worth investigating, where the objective is to minimise the cycle time given the number of workstations. Secondly, the stability of the assembly line with uncertain task times can also become an interesting and practical objective for optimisation, i.e. to optimise the chance that stations become overloaded given the cycle time and the number of stations. Consideration of a mixed-model line may also be of interest for future studies. In such a research, uncertain task times will change based on the product model being assembled. Therefore, the solution approach must also handle those variations to ensure an applicable line balance. Moreover, the ideas in this paper can be applied to other layout structures, including U-shaped TALBP and parallel lines.

Disclosure statement

No potential conflict of interest was reported by the author(s).

Funding

This work was supported in part by the National Natural Science Foundation of China [grant numbers 71901006 and 71772009], in part by the Base Project of the Beijing Social Science Foundation [grant number 19GLC053], and in part by the International Research Cooperation Seed Fund of the Beijing University of Technology [grant number 2018B24].

ORCID

Ibrahim Kucukkoc http://orcid.org/0000-0001-6042-6896

References

Agpak, K, M. Fatih Yegül, and H. Gökçen. 2012. "Two-sided U-type Assembly Line Balancing Problem." *International Journal of Production Research* 50 (18): 5035–5047.

Bartholdi, J. J. 1993. "Balancing Two-sided Assembly Lines: A Case Study." *International Journal of Production Research* 31 (10): 2447–2461.

Battaïa, O., and A. Dolgui. 2013. "A Taxonomy of Line Balancing Problems and Their Solutionapproaches." *International Journal of Production Economics* 142 (2): 259–277.

Becker, C., and A. Scholl. 2006. "A Survey on Problems and Methods in Generalized Assembly Line Balancing." *European Journal of Operational Research* 168 (3): 694–715.

Boysen, N., M. Fliedner, and A. Scholl. 2008. "Assembly Line Balancing: Which Model to Use when?" *International Journal of Production Economics* 111 (2): 509–528.

Cerqueus, A., and X. Delorme. 2019. "A Branch-and-bound Method for the Bi-objective Simple Line Assembly Balancing Problem." *International Journal of Production Research* 57 (18): 5640–5659.

Charnes, A., and W. W. Cooper. 1961. *Management Models and Industrial Applications of Linear Programming*. New York: Wiley.

Chiang, W.-C., T. L. Urban, and C. Luo. 2015. "Balancing Stochastic Two-sided Assembly Lines." *International Journal of Production Research* 54 (20): 6232–6250.

Delice, Y., E. K. Aydogan, and U. Özcan. 2016. "Stochastic Two-sided U-type Assembly Line Balancing: A Genetic Algorithm Approach." *International Journal of Production Research* 54 (11): 3429–3451.

Delice, Y., Emel Kizilkaya Aydogan, U. Özcan, and M. S. Ilkay. 2017a. "A Modified Particle Swarm Optimization Algorithm to Mixed-model Two-sided Assembly Line Balancing." *Journal of Intelligent Manufacturing* 28 (1): 23–36.

Delice, Y., Emel Kizilkaya Aydogan, U. Özcan, and M. S. Ilkay. 2017b. "Balancing Two-sided U-type Assembly Lines Using Modified Particle Swarm Optimization Algorithm." *4or* 15 (1): 37–66.

Dong, J., L. Zhang, T. Xiao, and H. Mao. 2014. "Balancing and Sequencing of Stochastic Mixed-model Assembly U-lines to Minimise the Expectation of Work Overload Time." *International Journal of Production Research* 52 (24): 7529–7548.

Gao, X., and L. Jia. 2016. "Degree-constrained Minimum Spanning Tree Problem with Uncertain Edge Weights." *Applied Soft Computing* 56: 580–588.

Gurevsky, E., O. Battaïa, and A. Dolgui. 2012. "Balancing of Simple Assembly Lines Under Variations of Task Processing Times." *Annals of Operations Research* 201 (1): 265–286.

Hazır, Ö., and A. Dolgui. 2012. "Assembly Line Balancing Under Uncertainty: Robust Optimization Models and Exact Solution Method." *Computers and Industrial Engineering* 65 (2): 261–267.

Hu, X., and C. Wu. 2018. "Workload Smoothing in Two-sided Assembly Lines." *Assembly Automation* 38 (1): 51–56.

Hu, X., E. Wu, J. Bao, and Y. Jin. 2010. "A Branch-and-bound Algorithm to Minimize the Line Length of a Two-sided Assembly Line." *European Journal of Operational Research* 206 (3): 703–707.

Hu, X., E. Wu, and Y. Jin. 2008. "A Station-oriented Enumerative Algorithm for Two-sided Assembly Line Balancing." *European Journal of Operational Research* 186 (1): 435–440.

Ke, H., H. Liu, and G. Tian. 2015. "An Uncertain Random Programming Model for Project Scheduling Problem." *International Journal of Intelligent Systems* 30 (1): 66–79.

Khorasanian, D., S. R. Hejazi, and G. Moslehi. 2013. "Two-sided Assembly Line Balancing Considering the Relationships Between Tasks." *Computers and Industrial Engineering* 66 (4): 1096–1105.

Kim, Y. K., Y. Kim, and Y. J. Kim. 2000. "Two-sided Assembly Line Balancing: A Genetic Algorithm Approach." *Production Planning & Control* 11 (1): 44–53.

Kim, Y. K., W. S. Song, and J. H. Kim. 2009. "A Mathematical Model and a Genetic Algorithm for Two-sided Assembly Line Balancing." *Computers and Operations Research* 36 (3): 853–865.

Kirkpatrick, S., C. D. Gelatt, and M. P. Vecchi. 1983. "Optimization by Simulated Annealing." *Science* 220 (4598): 671–680.

Kucukkoc, I. 2018. "A Nondominated Sorting Ant Colony Optimization Algorithm for Complex Assembly Line Balancing Problem Incorporating Incompatible Task Sets." *Pamukkale University Journal of Engineering Sciences* 24 (1): 141–152.

Kucukkoc, I., Z. Li, A. D. Karaoglan, and D. Z. Zhang. 2018. "Balancing of Mixed-model Two-sided Assembly Lines with Underground Workstations: A Mathematical Model and Ant Colony Optimization Algorithm." *International Journal of Production Economics* 205: 228–243.

Kucukkoc, I., and D. Z. Zhang. 2016. "Mixed-model Parallel Two-sided Assembly Line Balancing Problem: A Flexible Agent-based Ant Colony Optimization Approach." *Computers and Industrial Engineering* 97: 58–72.

Lai, T.-C., Y. N. Sotskov, and A. Dolgui. 2019. "The Stability Radius of An Optimal Line Balance with Maximum Efficiency for a Simple Assembly Line." *European Journal of Operational Research* 274 (2): 466–481.

Lai, T.-C., Y. N. Sotskov, A. Dolgui, and A. Zatsiupa. 2016. "Stability Radii of Optimal Assembly Line Balances with a Fixed Workstation Set." *International Journal of Production Economics* 182: 356–371.

Lee, T. O., Y. Kim, and Y. K. Kim. 2001. "Two-sided Assembly Line Balancing to Maximize Work Relatedness and Slackness." *Computers and Industrial Engineering* 40 (3): 273–292.

Lei, D., and X. Guo. 2016. "Variable Neighborhood Search for the Second Type of Two-sided Assembly Line Balancing Problem." *Computers and Operations Research* 72: 183–188.

Li, Z., I. Kucukkoc, and J. M. Nilakantan. 2017. "Comprehensive Review and Evaluation of Heuristics and Meta-heuristics for Two-sided Assembly Line Balancing Problem." *Computers and Operations Research* 84: 146–161.

Li, Z., I. Kucukkoc, and Q. Tang. 2019. "A Comparative Study of Exact Methods for the Simple Assembly Line Balancing Problem." *Soft Computing*. doi:10.1007/s00500-019-04609-9.

Li, Z., I. Kucukkoc, and Z. Zhang. 2020. "Branch, Bound and Remember Algorithm for Two-Sided Assembly Line Balancing Problem." *European Journal of Operational Research*. doi:10.1016/j.ejor.2020.01.032.

Li, R., and G. Liu. 2017. "An Uncertain Goal Programming Model for Machine Scheduling Problem." *Journal of Intelligent Manufacturing* 28 (3): 689–694.

Li, Z., Q. Tang, and L. P. Zhang. 2016. "Minimizing Energy Consumption and Cycle Time in Two-sided Robotic Assembly Line Systems Using Restarted Simulated Annealing Algorithm." *Journal of Cleaner Production* 135: 508–522.

Li, Z., Q. Tang, and L. P. Zhang. 2017. "Two-sided Assembly Line Balancing Problem of Type I: Improvements, a Simple Algorithm and a Comprehensive Study." *Computers and Operations Research* 79: 78–93.

Li, D., C. Zhang, X. Shao, and W. Lin. 2016a. "A Multi-objective TLBO Algorithm for Balancing Two-sided Assembly Line with Multiple Constraints." *Journal of Intelligent Manufacturing* 27 (4): 725–739.

Li, D., C. Zhang, G. Tian, X. Shao, and Z. Li. 2016b. "Multiobjective Program and Hybrid Imperialist Competitive Algorithm for the Mixed-Model Two-Sided Assembly Lines Subject to Multiple Constraints." *IEEE Transactions on Systems, Man, and Cybernetics: Systems* 48 (1): 1–11.

Li, B., and Y. Zhu. 2016. "Parametric Optimal Control for Uncertain Linear Quadratic Models." *Applied Soft Computing* 56: 543–550.

Lin, C., P. Jin, and Z. Bo. 2017. "Uncertain Goal Programming Models for Bicriteria Solid Transportation Problem." *Applied Soft Computing* 51: 49–59.

Liu, B. 2007. *Uncertainty Theory*. Berlin: Springer.

Liu, B. 2009a. "Some Research Problems in Uncertainty Theory." *Journal of Uncertain Systems* 3 (1): 3–10.

Liu, B. 2009b. *Theory and Practice of Uncertain Programming*. Berlin: Springer.

Liu, B. 2010. *Uncertainty Theory: A Branch of Mathematics for Modeling Human Uncertainty*. Berlin.Springer.

Ning, Y., and T. Su. 2017. "A Multilevel Approach for Modelling Vehicle Routing Problem with Uncertain Travelling Time." *Journal of Intelligent Manufacturing* 28 (3): 683–688.
Otto, A., C. Otto, and A. Scholl. 2013. "Systematic Data Generation and Test Design for Solution Algorithms on the Example of SALBPGen for Assembly Line Balancing." *European Journal of Operational Research* 228 (1): 33–45.
Özcan, U. 2010. "Balancing Stochastic Two-sided Assembly Lines: A Chance-constrained, Piecewise-linear, Mixed Integer Program and a Simulated Annealing Algorithm." *European Journal of Operational Research* 205: 81–97.
Roshani, A., and D. Giglio. 2016. "Simulated Annealing Algorithms for the Multi-manned Assembly Line Balancing Problem: Minimising Cycle Time." *International Journal of Production Research* 55 (10): 2731–2751.
Saif, U., Z. Guan, L. Zhang, J. Mirza, and Y. Lei. 2017. "Hybrid Pareto Artificial Bee Colony Algorithm for Assembly Line Balancing with Task Time Variations." *International Journal of Computer Integrated Manufacturing* 30 (2–3): 255–270.
Saif, U., Z. Guan, L. Zhang, F. Zhang, B. Wang, and J. Mirza. 2019. "Multi-objective Artificial Bee Colony Algorithm for Order Oriented Simultaneous Sequencing and Balancing of Multi-mixed Model Assembly Line." *Journal of Intelligent Manufacturing* 30 (3): 1195–1220.
Sivasankaran, P., and P. Shahabudeen. 2014. "Literature Review of Assembly Line Balancing Problems." *The International Journal of Advanced Manufacturing Technology* 73 (9–12): 1665–1694.
Sotskov, Y. N., A. Dolgui, T. -C. Lai, and A. Zatsiupa. 2015. "Enumerations and Stability Analysis of Feasible and Optimal Line Balances for Simple Assembly Lines." *Computers & Industrial Engineering* 90: 241–258.
Sotskov, Y. N., A. Dolgui, and M. -C. Portmann. 2006. "Stability Analysis of An Optimal Balance for An Assembly Line with Fixed Cycle Time." *European Journal of Operational Research* 168 (3): 783–797. Balancing Assembly and Transfer lines.
Tang, Q., Z. Li, and L. Zhang. 2016. "An Effective Discrete Artificial Bee Colony Algorithm with Idle Time Reduction Techniques for Two-sided Assembly Line Balancing Problem of Type-II." *Computers and Industrial Engineering* 97: 146–156.
Tang, Q., Z. Li, L. Zhang, and C. Zhang. 2017. "Balancing Stochastic Two-sided Assembly Line with Multiple Constraints Using Hybrid Teaching-learning-based Optimization Algorithm." *Computers and Operations Research* 82: 102–113.
Tuncel, G., and D. Aydin. 2014. "Two-sided Assembly Line Balancing Using Teaching-learning Based Optimization Algorithm." *Computers and Industrial Engineering* 74 (1): 291–299.
Vicente, J. D., J. Lanchares, and R. Hermida. 2003. "Placement by Thermodynamic Simulated Annealing." *Physics Letters A* 317 (5): 415–423.
Wen, M., Z. Qin, and R. Kang. 2014. "The α -cost Minimization Model for Capacitated Facility Location-allocation Problem with Uncertain Demands." *Fuzzy Optimization and Decision Making* 13 (3): 345–356.
Yang, W., and W. Cheng. 2019. "Modelling and Solving Mixed-model Two-sided Assembly Line Balancing Problem with Sequence-dependent Setup Time." *International Journal of Production Research* 1–22. doi:10.1080/00207543.2019.168325
Zhang, Y., X. Hu, and C. Wu. 2018. "A Modified Multi-objective Genetic Algorithm for Two-sided Assembly Line Re-balancing Problem of a Shovel Loader." *International Journal of Production Research* 56 (9): 3043–3063.
Zhang, Y., X. Hu, and C. Wu. 2019. "Improved Imperialist Competitive Algorithms for Rebalancing Multi-objective Two-sided Assembly Lines with Space and Resource Constraints." *International Journal of Production Research*: 1–29. doi:10.1080/00207543.2019.1633023
Zhu, Y. 2012. "Functions of Uncertain Variables and Uncertain Programming." *Journal of Uncertain Systems* 6 (4): 278–288.

Appendix

In the appendix, we introduce some fundamental concepts and properties in uncertainty theory including uncertain measure, uncertain variable and uncertain programming.

Definition 1 (Liu 2007) Let Ł be a σ-algebra on a non-empty set Γ. A set function $\mathcal{M} : \mathcal{L} \to [0, 1]$ is called an uncertain measure if it satisfies the following axioms,

Axiom 1. $\mathcal{M}\{\Gamma\} = 1$ *for the universal set* Γ*;*
Axiom 2. $\mathcal{M}\{\Lambda\} + \mathcal{M}\{\Lambda^c\} = 1$ *for any event* Λ*;*
Axiom 3. For every countable sequence of events $\Lambda_1, \Lambda_2, \ldots$*, we have*

$$\mathcal{M}\left\{\bigcup_{i=1}^{\infty} \Lambda_i\right\} \leq \sum_{i=1}^{\infty} \mathcal{M}\{\Lambda_i\}.$$

In order to provide the operational law, Liu (2009a) defined the product uncertain measure on the product σ-algebra Ł, called product axiom.

Axiom 4. Let $(\Gamma_k, \mathcal{L}_k, \mathcal{M}_k)$ *be uncertainty spaces for* $k = 1, 2, \ldots$*. The product uncertain measure* $\mathcal{M}$ *is an uncertain measure satisfying*

$$\mathcal{M}\left\{\prod_{k=1}^{\infty} \Lambda_k\right\} = \bigwedge_{k=1}^{\infty} \mathcal{M}_k\{\Lambda_k\}$$

where Λ_k *are arbitrarily chosen events from* $\mathcal{L}_k$ *for* $k = 1, 2, \ldots$*, respectively.*

Definition 2 (Liu 2007) An uncertain variable ξ is a function from an uncertainty space $(\Gamma, \mathcal{L}, \mathcal{M})$ to the set of real numbers such that for any Borel set B of real numbers, the set

$$\{\xi \in B\} = \{\gamma \in \Gamma \mid \xi(\gamma) \in B\}$$

is an event.

In order to describe uncertain variable in practice, uncertainty distribution $\Phi : \Re \to [0, 1]$ of an uncertain variable ξ is defined as $\Phi(x) = \mathcal{M}\{\xi \leq x\}$. An uncertainty distribution $\Phi(x)$ is said to be regular if it is a continuous and strictly increasing function with respect to x at which $0 < \Phi(x) < 1$, and

$$\lim_{x \to -\infty} \Phi(x) = 0, \quad \lim_{x \to +\infty} \Phi(x) = 1.$$

If ξ is an uncertain variable with regular uncertainty distribution Φ, then we call the inverse function $\Phi^{-1}(\alpha)$ as the inverse uncertainty distribution of ξ.

An uncertain variable ξ is called normal if it has a normal uncertainty distribution

$$\Phi(x) = \left(1 + \exp\left(\frac{\pi(e - x)}{\sqrt{3}\sigma}\right)\right)^{-1},$$

denoted by $\mathcal{N}(e, \sigma)$, where e and σ are real numbers with $\sigma > 0$. The inverse uncertainty distribution of normal uncertain variable $\mathcal{N}(e, \sigma)$ is

$$\Phi^{-1}(\alpha) = e + \frac{\sigma\sqrt{3}}{\pi} \ln \frac{\alpha}{1 - \alpha}.$$

Definition 3 (Liu 2009a) The uncertain variables $\xi_1, \xi_2, \ldots, \xi_m$ are said to be independent if

$$\mathcal{M}\left\{\bigcap_{i=1}^{m} \{\xi_i \in B_i\}\right\} = \bigwedge_{i=1}^{m} \mathcal{M}\{\xi_i \in B_i\}$$

for any Borel sets $B_1, B_2, \ldots, B_m$ of real numbers.

The expected value operator of uncertain variable, proposed by Liu (2007), is the average value of uncertain variable in the sense of uncertain measure, and represents the size of uncertain variable.

Definition 4 (Liu 2007) Let ξ be an uncertain variable. Then the expected value of ξ is defined as

$$\mathrm{E}[\xi] = \int_0^{+\infty} \mathcal{M}\{\xi \geq x\}\,\mathrm{d}x - \int_{-\infty}^{0} \mathcal{M}\{\xi \leq x\}\,\mathrm{d}x$$

provided that at least one of the two integrals is finite.

Further, for an uncertain variable ξ with uncertainty distribution $\Phi(x)$, Liu (2010) showed that its expected value can be obtained by

$$\mathrm{E}[\xi] = \int_{-\infty}^{+\infty} x\,\mathrm{d}\Phi(x). \tag{A1}$$

Furthermore, if $\Phi(x)$ is regular, then

$$\mathrm{E}[\xi] = \int_0^1 \Phi^{-1}(\alpha)\,\mathrm{d}\alpha. \tag{A2}$$

THEOREM 1 (Liu 2009a) *Let ξ and η be independent uncertain variables with finite expected values. Then for any real numbers a and b, we have*

$$\mathrm{E}[a\xi + b\eta] = a\mathrm{E}[\xi] + b\mathrm{E}[\eta].$$

THEOREM 2 (Liu 2010) *Assume $\xi_1, \xi_2, \ldots, \xi_n$ are independent uncertain variables with regular uncertainty distributions $\Phi_1, \Phi_2, \ldots, \Phi_n$, respectively. If $f(\xi_1, \xi_2, \ldots, \xi_n)$ is strictly increasing with respect to $\xi_1, \xi_2, \ldots, \xi_n$, then the uncertain variable $\xi = f(\xi_1, \xi_2, \ldots, \xi_n)$ has an inverse distribution*

$$\Psi^{-1}(\alpha) = f(\Phi_1^{-1}(\alpha), \Phi_2^{-1}(\alpha), \ldots, \Phi_n^{-1}(\alpha)).$$

THEOREM 3 (Liu 2010) *Let $\xi_1, \xi_2, \ldots, \xi_n$ be independent uncertain variables with regular uncertainty distributions $\Phi_1, \Phi_2, \ldots, \Phi_n$, respectively. If a function $f(x, \xi_1, \xi_2, \ldots, \xi_n)$ is strictly increasing with respect to $\xi_1, \xi_2, \ldots, \xi_m$ and strictly decreasing with respect to $\xi_{m+1}, \xi_{m+2}, \ldots, \xi_n$, then the constraint*

$$\mathcal{M}\{f(x, \xi_1, \xi_2, \ldots, \xi_n) \leq 0\} \geq \alpha$$

holds if and only if,

$$f(\Phi_1^{-1}(\alpha), \Phi_2^{-1}(\alpha), \ldots, \Phi_m^{-1}(\alpha), \Phi_{m+1}^{-1}(1 - \alpha), \Phi_{m+2}^{-1}(1 - \alpha), \ldots, \Phi_n^{-1}(1 - \alpha)) \leq 0.$$

Computational results on the problems generated by SALBPGen

Table A1. Computational results on the problems from Otto's datasets (1000 tasks)-low task variance.

			$\alpha = 0.975$				$\alpha = 0.9$		
Problem	c	LB	GA NM[WS]	TLBO NM[WS]	SA NM[WS]	LB	GA NM[WS]	TLBO NM[WS]	SA NM[WS]
P1000_1	7400	12[23]	12[24]	12[24]	12[**23**]	11[21]	11[22]	11[22]	11[22]
	9600	9[18]	9[18]	9[18]	9[18]	9[17]	9[18]	9[18]	9[**17**]
	12000	7[14]	7[14]	7[14]	7[14]	7[13]	7[14]	7[14]	7[**13**]
	16000	6[11]	6[12]	6[12]	6[**11**]	5[10]	5[10]	5[10]	5[10]
	19000	5[9]	5[10]	5[10]	5[**9**]	5[9]	5[10]	5[10]	5[**9**]
P1000_2	7400	12[23]	12[24]	12[24]	12[**23**]	11[21]	11[22]	11[22]	11[22]
	9600	9[18]	9[18]	9[18]	9[18]	9[17]	9[18]	9[18]	9[**17**]
	12000	7[14]	7[14]	7[14]	7[14]	7[13]	7[14]	7[14]	7[**13**]
	16000	6[11]	6[12]	6[12]	6[**11**]	5[10]	5[10]	5[10]	5[10]
	19000	5[9]	5[10]	5[10]	5[**9**]	5[9]	5[10]	5[10]	5[**9**]
P1000_3	7400	12[23]	12[24]	12[24]	12[**23**]	11[21]	11[22]	11[22]	11[22]
	9600	9[18]	9[18]	9[18]	9[18]	9[17]	9[18]	9[18]	9[**17**]
	12000	7[14]	7[14]	7[14]	7[14]	7[13]	7[14]	7[14]	7[**13**]
	16000	6[11]	6[12]	6[12]	6[**11**]	5[10]	5[10]	5[10]	5[10]
	19000	5[9]	5[10]	5[10]	5[**9**]	5[9]	5[10]	5[10]	5[**9**]

Note: Best results in bold.

Table A2. Computational results on the problems from Otto's datasets (1000 tasks)-high-task variance.

			$\alpha = 0.975$				$\alpha = 0.9$		
Problem	c	LB	GA NM[WS]	TLBO NM[WS]	SA NM[WS]	LB	GA NM[WS]	TLBO NM[WS]	SA NM[WS]
P1000_1	7400	14[27]	14[28]	14[28]	14[28]	12[24]	13[26]	**12[24]**	13[25]
	9600	11[21]	11[22]	11[22]	11[**21**]	10[19]	10[10]	10[10]	10[**19**]
	12000	9[17]	9[18]	9[18]	9[**17**]	8[15]	8[16]	8[16]	8[**15**]
	16000	7[13]	7[14]	7[14]	7[**13**]	6[11]	6[12]	6[12]	6[**11**]
	19000	6[11]	6[12]	6[12]	6[**11**]	5[10]	5[10]	5[10]	5[10]
P1000_2	7400	14[27]	14[28]	14[28]	14[28]	19[37]	13[26]	12[24]	12[24]
	9600	11[21]	11[22]	11[22]	11[**21**]	15[30]	10[10]	10[10]	10[**19**]
	12000	9[17]	9[18]	9[18]	9[**17**]	13[25]	8[16]	8[16]	8[**15**]
	16000	7[13]	7[14]	7[14]	7[**13**]	11[21]	6[12]	6[12]	6[**11**]
	19000	6[11]	6[12]	6[12]	6[**11**]	10[19]	5[10]	5[10]	5[10]
P1000_3	7400	14[27]	14[28]	14[28]	14[28]	9[17]	13[26]	12[24]	12[24]
	9600	11[21]	11[22]	11[22]	11[**21**]	8[15]	10[10]	10[10]	10[**19**]
	12000	9[17]	9[18]	9[18]	9[**17**]	16[31]	8[16]	8[16]	8[**15**]
	16000	7[13]	7[14]	7[14]	7[**13**]	13[26]	6[12]	6[12]	6[**11**]
	19000	6[11]	6[12]	6[12]	6[**11**]	12[23]	5[10]	5[10]	5[10]

Note: Best results in bold.